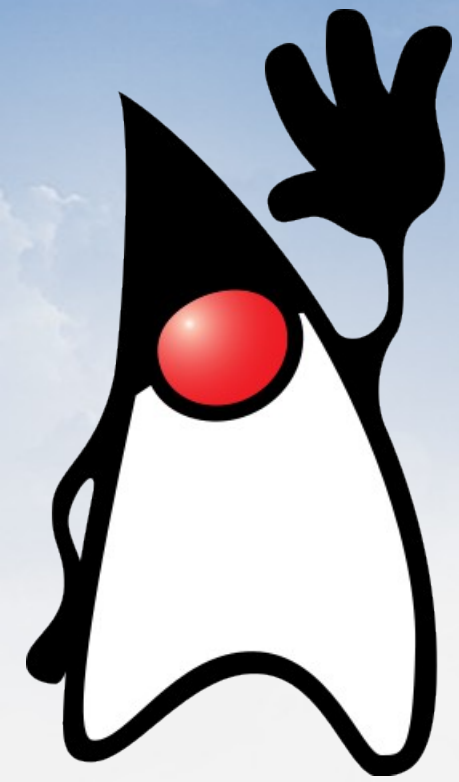




Object Oriented Programming in Java

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

From the beginning

- History of programming paradigms
 - GoTo Programming
(spaghetti code!)
 - Structured Programming
 - Object-Oriented Programming
 - ??? *Event-driven programming*
- response to need to
 - build ever larger programs
- correctly, on time & on budget

[1950 - 1970]

[1970 - 1990]

[1990 – today?]

- Describe a particular object
 - A bar of soap with rectangular prism shape (5" x 2" x 4") with olive oil aroma, bubbling
- Describe a particular car
 - 1957 Chevrolet Bel air with 2 doors, 4 wheels, 3,859 cc engine, red, running
- Notice how we describe objects by some category/property values



OOP Terms

- **Class** - Blueprint/template mold
 - Properties/states
 - Functionality/Services
(examines/alters state/properties)
- **object** - Individual/unique thing : Instance
(an instance of a class)
 - Particular value for each property/state
 - & functionality of all members of class.

Class

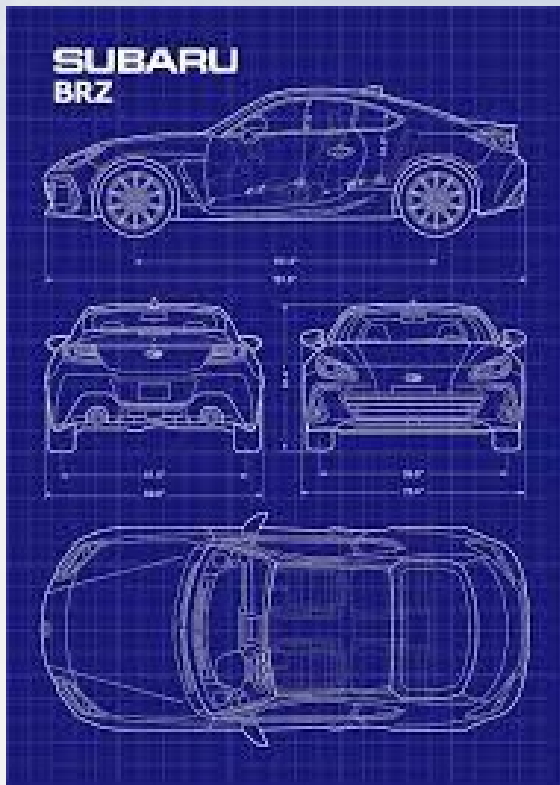


Object

If you want a rose aroma soap, you can create from the mold



Class



Object



If you want a red car, you can create from the blueprint

Soap

aroma
length
height
width
isBubbling

bubble
smell

aroma: Aloe Vera
length: 4
height: 4
width: 1
isBubbling: No

ClassName

property
property
property
state
state

methods

Car

modelYear
brand
engine
color

run
stop
roar

modelYear: 2027
brand: Toyota
engine: 3800
color: Blue

Writing the Car Class/Blueprint

```
package myworld;
// Car - simple example only!
// Author: Safa

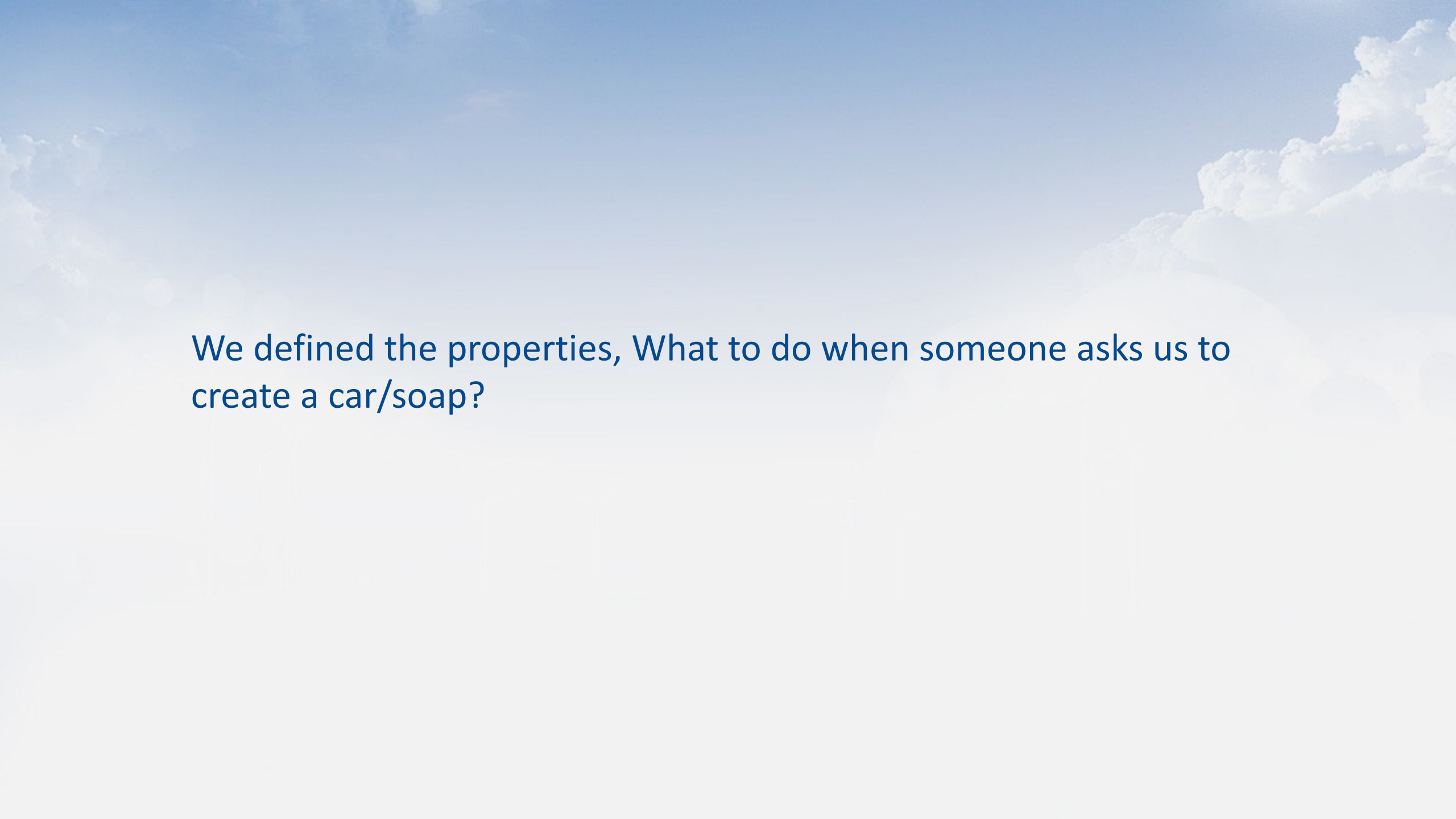
public class Car {

    // properties
    String      brand;
    int         year;
    boolean     isRunning;

    // constructors

    // methods
    public void run() {
        isRunning=true;
    }

    public void roar() {
        System.out.println( brand + " car is roaring" );
    }
}
```



We defined the properties, What to do when someone asks us to create a car/soap?

Constructor method

- It creates connection between the given arguments and class variables.
- It is the policy for creating new object instances
- Constructor method should have the same name with class
- **DOES NOT HAVE A RETURN TYPE**
- Can have arguments/no argument
- No argument constructor must be implemented.
- The file name should match with the class name

Creating the class

```
package myworld;
// Car - simple example only!
// Author: Safa

public class Car {

    // properties
    String brand;
    int year;
    boolean isRunning;

    // constructors
    public Car( String thebrand, int theYear) {
        brand = theBrand;
        year = theYear;
        isRunning = false;
    }

    // methods
    public void run() {
        isRunning = true;
    }

    public void roar() {
        System.out.println( brand + " car is roaring" );
    }
}
```

```
public Car(
    String thebrand,
    int theYear) {
    brand = theBrand;
    year = theYear;
    isRunning = false;
}
```

when a new object is created with brand and year parameters, it will be not running by default.

```

package myworld;
// Car - simple example only!
// Author: Safa
public class Car {

    // properties
    String brand;
    int    year;
    boolean isRunning;

    // constructors
    public Car( String theBrand, int theYear) {
        brand = theBrand;
        year = theYear;
        isRunning=false;
    }

    // methods
    public void run() {
        isRunning=true;
    }

    public void roar() {
        if(isRunning){
            System.out.println( brand + " car is roaring" );
        }else{
            System.out.println(brand+ " car is not running");
        }
    }
}

```

```

package myworld;
// CarTest - demo Car class
// Author: Safa

public class CarTest {
    public static void main( String[] args) {
        // VARIABLES
        Car myCar;
        Car yourCar;

        // PROGRAM CODE
        myCar = new Car( "Toyota", 2000);
        yourCar = new Car( "Subaru", 2023);

        myCar.run();

        yourCar.roar();
        myCar.roar();
    }
} // end of class CarTest

```

Subaru car is not running
Toyota car is roaring

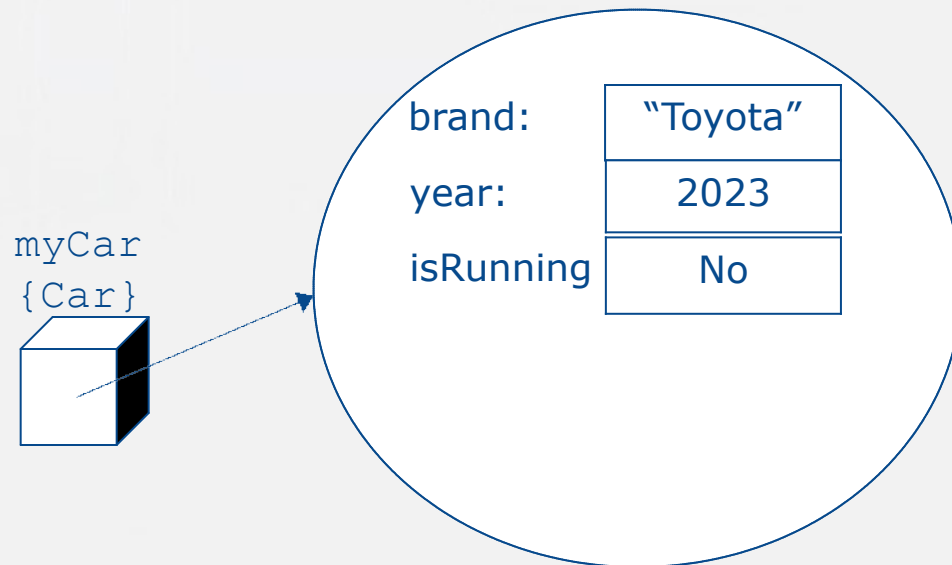
Creating the object

- Always
 - Declare variable to “hold” object
 - Create object using “new” statement
 - Call object’s methods

```
Car myCar;
```

```
myCar=new Car(“Toyota”,2023);
```

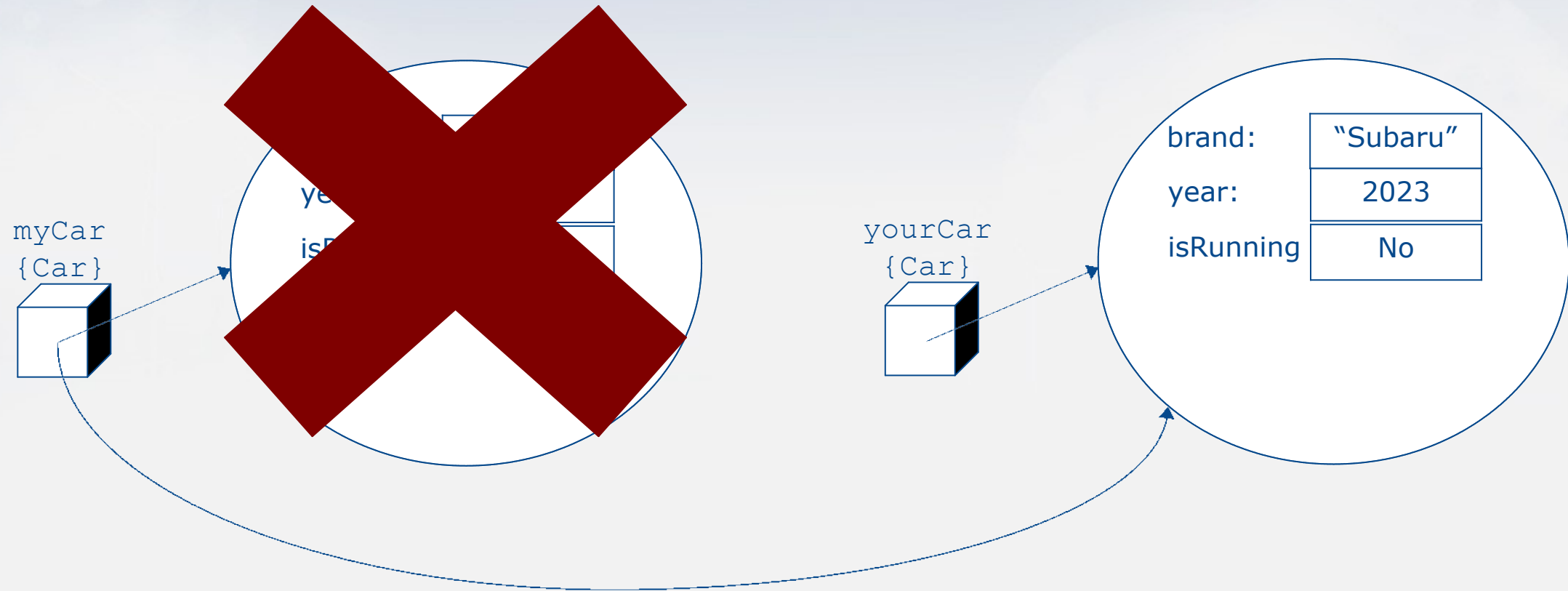
```
myCar.roar();
```



Put this in method
of another class,
(e.g main
method)

IMPORTANT OBJECT REFERENCE PROBLEM

```
Car myCar;  
myCar=new Car( "Toyota" , 2000);  
Car yourCar;  
yourCar=new Car( "Subaru" , 2023);  
  
myCar=yourCar;
```



```

package myworld;
// Car - simple example only!
// Author: Safa

public class Car {

    // properties
    String    brand;
    int       year;
    boolean   isRunning;

    // constructors
    public Car( String theBrand, int theYear) {
        brand = theBrand;
        year = theYear;
        isRunning = false;
    }

    // methods
    public void run() {
        isRunning = true;
    }

    public void roar() {
        if(isRunning){
            System.out.println( brand + " car is roaring" );
        }else{
            System.out.println(brand+ " car is not
running");
        }
    }
}

```

```

package myworld;
// CarTest - demo Car class
// Author: Safa
public class CarTest {
    public static void main( String[] args) {
        // VARIABLES
        Car    myCar;
        Car    yourCar;

        // PROGRAM CODE
        myCar = new Car( "Toyota", 2000);
        yourCar = new Car( "Subaru", 2023);

        myCar.run();
        myCar.roar();

        myCar=yourCar;
        System.out.println("After the cars are equal");
        myCar.roar();
        yourCar.roar();
    }
} // end of class CarTest

```

Toyota car is roaring
 After the cars are equal
 Subaru car is not running
 Subaru car is not running

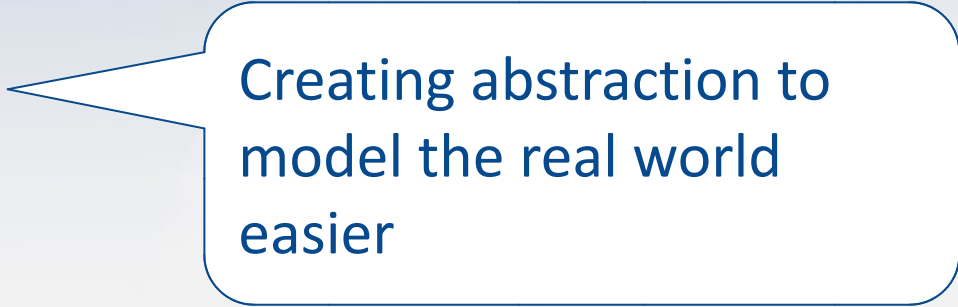
Why is the use of it?

■ Ease of reuse

- Speeds implementation & facilitates maintenance.
- Component-based approach
 - Easy to use existing code modules
 - Easy to modify code to fit circumstances!

■ A natural way to view/model world

- Makes design quicker, easier & less error-prone.



Creating abstraction to model the real world easier

```

package myworld;
// Car - simple example only!
// Author: Safa
public class Car {
    // properties
    String    brand;
    int       year;
    boolean   isRunning;

    // different constructors
    public Car( String theBrand, int theYear) {
        brand = theBrand;
        year = theYear;
        isRunning = false;
    }

    public Car() {
        brand = "Toyota";
        year = 1980;
        isRunning = false;
    }

    public Car(boolean runState) {
        brand = "Toyota";
        year = 1980;
        isRunning = runState;
    }
}

```

```

package myworld;
// CarTest - demo Car class
// Author: Safa

public class CarTest {
    public static void main( String[] args) {
        // VARIABLES
        Car    myCar;
        Car    yourCar;

        // PROGRAM CODE
        myCar = new Car( "Toyota", 2000);
        yourCar = new Car();
        Car anotherone= new Car(true);
        System.out.print(yourCar.year); // 1980
    }

} // end of class CarTest

```

We can access the instance variables by **.variable**

yourCar.year will give us the year instance variable of the car